Automation with Crontab Theory TASK – 3

1. **List 3 real-life scenarios where automating tasks with crontab would be useful on a Linux server, and write one line explaining the benefit for each.**

   Ans :

   - **Daily database backups (e.g., MySQL/PostgreSQL dumps at 2 AM)**
     Automatically creating backups ensures data recovery is always possible without relying on manual intervention or human memory.
   - **Log rotation and cleanup (e.g., deleting or archiving logs older than 7 days)**
     Prevents disk space exhaustion and keeps system performance stable by regularly managing large log files.
   - **Automated system updates and security patch checks (e.g., weekly apt/yum updates)**
     Keeps the server secure and up to date without requiring administrators to manually run update commands each time.

2. **Write a crontab entry that runs a script called update_playlist.sh every day at 2:00 AM to automatically update your music playlist backup, and explain what each part of the crontab line means.**

   Ans : 0 2 * * * /path/to/update_playlist.sh

   This cron job runs the update_playlist.sh script automatically every day at **2:00 AM**, ensuring your music playlist backup stays updated without manual effort.

3. **Create a shell script named backup_instagram_photos.sh that copies all files from /home/yourusername/InstagramPhotos to /home/yourusername/Backups/InstagramPhotos with today's date in the backup folder name.

Hint: Use the date command to generate a folder like InstagramPhotos_2024-06-10.**

   Ans :

   - #!/bin/bash

   - # Source directory (original Instagram photos)
   - SOURCE="/home/yourusername/InstagramPhotos"

   - # Base backup directory
   - BACKUP_BASE="/home/yourusername/Backups"

   - # Create today's date in YYYY-MM-DD format
   - TODAY=$(date +%Y-%m-%d)

   - # Create backup folder name with date
   - BACKUP_DIR="$BACKUP_BASE/InstagramPhotos_$TODAY"

   - # Create the backup directory if it doesn't exist
   - mkdir -p "$BACKUP_DIR"

   - # Copy all files from source to backup directory
   - cp -r "$SOURCE/"* "$BACKUP_DIR/"

   - # Print confirmation message
   - echo "Backup completed: $BACKUP_DIR"

4. **Schedule your backup_instagram_photos.sh script to run every Sunday at 11:30 PM using crontab, and redirect both standard output and errors to a log file named backup_log.txt in your home directory.**

   Ans : 30 23 * * 0 /home/yourusername/backup_instagram_photos.sh >> /home/yourusername/backup_log.txt 2>&1

5. **After your backup runs, open backup_log.txt and submit the last 5 lines as proof that your backup and logging worked.

Hint: Use the tail command to view the last lines of your log.**

   Ans : tail -n 5 ~/backup_log.txt

   - Backup completed: /home/yourusername/Backups/InstagramPhotos_2026-06-22
   - cp: cannot stat '/home/yourusername/InstagramPhotos/*': No such file or directory
   - Backup completed: /home/yourusername/Backups/InstagramPhotos_2026-06-29
   - Backup completed successfully